极速版机器学习

作者：哈利波特 👑

1. 线性回归和逻辑回归

线性回归

建模：

线性回归模型假设因变量（目标变量）与自变量（特征变量）之间存在线性关系，同时引入一个**干扰项（误差项）**来反映模型未能解释的部分。具体模型形式如下：

$$y^{(i)} = \beta_0 + \beta_1 x_1^{(i)} + \beta_2 x_2^{(i)} + \cdots + \beta_n x_n^{(i)} + \epsilon^{(i)}$$

目标函数（最小二乘法）：

a. 最小二乘法（Ordinary Least Squares, OLS）

目标：最小化预测值与真实值之间的**均方误差（Mean Squared Error, MSE）**。

$$J(\beta) = \frac{1}{m}\sum_{i=1}^{m}\left(y^{(i)} - \hat{y}^{(i)}\right)^2 = \frac{1}{m}\sum_{i=1}^{m}\left(y^{(i)} - (\beta_0 + \beta_1 x_1^{(i)} + \cdots + \beta_n x_n^{(i)})\right)^2$$

其中 m 是样本数量，$\hat{y}^{(i)}$ 是第 i 个样本的预测值。

参数更新：

b. 梯度下降法（Gradient Descent）

当特征数量较多或数据规模较大时，通常采用迭代优化方法，如梯度下降法，来估计参数。

更新公式：

$$\beta_j := \beta_j - \alpha \frac{\partial J(\beta)}{\partial \beta_j}$$

偏导数计算：

对于线性回归，目标函数对 β_j 的偏导数为：

$$\frac{\partial J(\beta)}{\partial \beta_j} = -\frac{2}{m}\sum_{i=1}^{m}\left(y^{(i)} - \hat{y}^{(i)}\right)x_j^{(i)}$$

迭代步骤：

1. 初始化参数 β（可以设为零或随机值）。
2. 计算梯度 $\nabla J(\beta)$。
3. 更新参数 β：

$$\beta := \beta - \alpha \nabla J(\beta)$$

4. 重复步骤2和3，直到收敛（即参数更新幅度小于设定阈值）。

数据集形式：

y 是一列有 n 个观测值的观测变量，或者直接说因变量以便于理解，
X 是由多列特征组成的特征空间，假设有 p 列特征，简单理解就是有 p 个自变量，每个特征都有 n 个值，这与 y 是对应的：

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} \quad , \quad X = \begin{pmatrix} X_1^T \\ X_2^T \\ \vdots \\ X_n^T \end{pmatrix} = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1p} \\ 1 & x_{21} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \cdots & x_{np} \end{pmatrix}$$

β 是系数向量， ε 是干扰项（disturbanceterm），或称错误项（errorterm）：

$$\beta = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_p \end{pmatrix} \quad , \quad \varepsilon = \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{pmatrix}$$

梯度下降法：

- 简单容易实现
- 可能陷入局部最优
- 对参数尺度敏感（需要归一化）

为什么需要空间转换

因为线性回归模型是假设X和Y是线性的，但是现实中不一定所有都是线性的，所以进行空间转换，希望能更好的捕捉希望将这种非线性转换成线性关系。

了解偏差 误差 方差

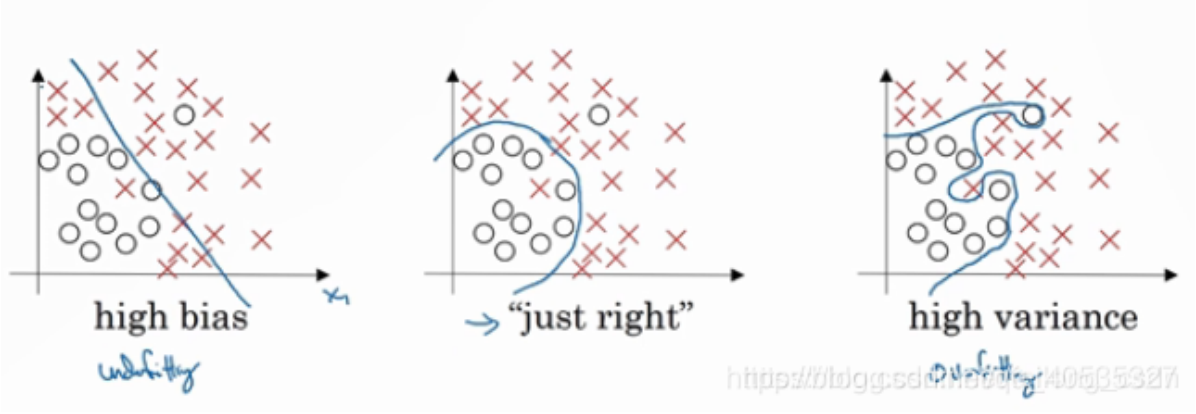
- **偏差**：总是低估或高估价格（预测值与真实值有系统性差异）。
- **误差**：每次预测与实际房价的差异（具体的预测偏差）。
- **方差**：不同训练集上的预测结果波动（模型的稳定性）。
- [机器学习中偏差、方差的理解与总结_机器学习 偏差 方差-CSDN博客]

平衡偏差与方差

在模型训练过程中，**偏差**和**方差**之间存在一种权衡：

- **高偏差**通常意味着模型太简单，无法捕捉到数据的复杂关系，可能导致欠拟合。
- **高方差**则意味着模型太复杂，过度依赖训练数据的细节，容易导致过拟合。

正则化项通过限制模型的复杂度，可以适度地增加偏差（使得模型更加简单），从而减少方差（降低模型对训练数据的过度拟合），这有助于提高模型在新数据上的泛化能力。



逻辑回归

逻辑回归就是在线性回归的基础上，增加了sigmoid函数，使得它的输出是 0~1 。

逻辑回归：模型假设目标变量是特征的线性组合，通过**Sigmoid函数**将其转化为一个概率值，形式为：

$$\hat{y} = \sigma(w_1x_1 + w_2x_2 + \cdots + w_nx_n + b) = \frac{1}{1 + e^{-(w^Tx+b)}}$$

这里的 $\hat{y}$ 是0到1之间的概率，表示样本属于某一类的概率。

与线性回归差别

- 线性回归是预测连续值的（房价），逻辑回归是进行分类的。
- 线性回归要求X和Y是线性关系，逻辑回归没有要求。
- 线性回归要求因变量（Y）是连续的，逻辑回归要求Y是离散（标签）。

目标函数：

逻辑回归：使用**交叉熵损失**（log loss），衡量预测的概率与真实标签之间的差异。其损失函数形式为：

$$L = -\frac{1}{m}\sum_{i=1}^{m}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

其中 $y_i \in \{0,1\}$ 是真实标签，$\hat{y}_i$ 是模型预测的概率。

参数更新

梯度更新规则：

$$w_j = w_j - \alpha \frac{\partial L}{\partial w_j}$$

其中梯度计算稍有不同，具体为：

$$\frac{\partial L}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)x_{ij}$$

为什么要增加正则项

逻辑回归：同样可以添加L1或L2正则化项，目标函数变为：

$$L_{\text{reg}} = -\frac{1}{m}\sum_{i=1}^{m}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)] + \lambda \sum_{j=1}^{n} w_j^2$$

其中 λ 是正则化参数。正则化项可以帮助减少过拟合，特别是在特征多且数据量较小时。

平衡偏差与方差

在模型训练过程中，**偏差**和**方差**之间存在一种权衡：

- **高偏差**通常意味着模型太简单，无法捕捉到数据的复杂关系，可能导致欠拟合。
- **高方差**则意味着模型太复杂，过度依赖训练数据的细节，容易导致过拟合。

正则化项通过限制模型的复杂度，可以适度地增加偏差（使得模型更加简单），从而减少方差（降低模型对训练数据的过度拟合），这有助于提高模型在新数据上的泛化能力。

提高分类模型的泛化能力。

2. 评判指标

看作业

3. PCA（主成分分析）

将数据从高维空间映射到低维空间，同时尽可能保留数据的主要特征（或信息）。非监督。

可以从**最小化重构误差理解**，或者从**最大化方差理解**。

- 最小化重构误差

  PCA通过选择最重要的方向来表示数据，使得通过这些方向来重建原始数据时，重构误差最小。

- 最大化方差

  通过将数据投影到这些方差最大的方向上，我们最大化了数据中保留的信息。

假设方向向量（主成分）是**两两正交的**。

- 这样就没有冗余信息，两两独立。

基本思想：PCA确实是通过寻找一组正交方向，然后将数据投影到这些方向上来实现降维的。

Theorem: Under the given M orthogonal directions $\boldsymbol{u}_i$, the *best approximation* to a data sample $\boldsymbol{x}$ is

$$\tilde{\boldsymbol{x}} = \alpha_1 \boldsymbol{u}_1 + \alpha_2 \boldsymbol{u}_2 + \cdots + \alpha_M \boldsymbol{u}_M$$

with the α_i equals to

$$\alpha_i = \boldsymbol{u}_i^T \boldsymbol{x}$$

点x在方向ui的投影

α_i的取值是由下面推导出来的（可以理解成是X在方向向量上的投影）：

Proof: 希望重构x与x尽可能一样

$$\|\boldsymbol{x} - \tilde{\boldsymbol{x}}\|^2 = \left\| \boldsymbol{x} - \sum_{i=1}^{M} \alpha_i \boldsymbol{u}_i \right\|^2$$

$$= \|\boldsymbol{x}\|^2 - 2\sum_{i=1}^{M} \alpha_i \boldsymbol{u}_i^T \boldsymbol{x} + \sum_{i=1}^{M} \alpha_i^2$$

因为向vec是正交

where we used $\boldsymbol{u}_i^T \boldsymbol{u}_j = 0$ for $i \neq j$ and 1 for $i = j$

二次

This is a quadratic function, and can be minimized when $\alpha_i = \boldsymbol{u}_i^T \boldsymbol{x}$

求偏导并假设为0

方向向量是基于协方差矩阵特征值分解，找到特征值最大的几个向量作为方向向量。再将X投影到这些方向向量上进行降维。

因为特征值越大，代表这个方向上的方差越大，即信息越多。

换个角度想，PCA也可以用作提取特征。

应用：

- 图像压缩
- 降噪
- 提取特征（像人脸识别）

缺陷：

- 适合于线性数据，不适合非线性数据。（想想看建模）

- 计算协方差矩阵对于尺度敏感，需要归一化。
- 需要大量的矩阵运算和特征值分解。

推导过程：

从最小化重构误差的角度理解 PCA

- 高维空间中的正交方向是一组两两正交的（单位）向量组，**正交可以表示成向量内积等于 0。**
- **定理：在给定 M 个标准正交方向 u_i 下，与数据样本 x 的最佳近似为：**
  $\hat{\mathbf{x}} = \alpha_1\mathbf{u}_1 + \alpha_2\mathbf{u}_2 + \ldots + \alpha_M\mathbf{u}_M$，$\alpha_i = \mathbf{u}_i^T\mathbf{x}$**（系数 α_i 可以算出，但最佳方向还未确定）。**
- 目标：确定最佳方向
  - 给定数据 $\{\mathbf{x}^{(n)}\}_{n=1}^N \in \mathbb{R}^D$，找出最能代表原始数据的正交方向 u_i，即 $\mathbf{x}^{(n)} \approx \sum_{i=1}^M \alpha_i^{(n)}\mathbf{u}_i$
  - 目标可以表述为最小化数据 $\mathbf{x}^{(n)}$ 与其近似值 $\hat{\mathbf{x}}^{(n)} = \sum_{i=1}^M \alpha_i^{(n)}\mathbf{u}_i$ 之间的误差
- 推导步骤：
  1. 中心化样本：$\mathbf{x}^{(n)} - \overline{\mathbf{x}} = \mathbf{x}^{(n)} - \frac{1}{N}\sum_{n=1}^N \mathbf{x}^{(n)}$
  2. 描述目标问题：

$$E = \frac{1}{N}\sum_{n=1}^N ||(\mathbf{x}^{(n)} - \overline{\mathbf{x}}) - \hat{\mathbf{x}}^{(n)}||^2$$
$$\alpha_I^{(n)} = \mathbf{u}_i^T(\mathbf{x}^{(n)} - \overline{\mathbf{x}})$$

  3. 将 $\hat{\mathbf{x}}^{(n)}$ 的表达式代入 E，并将平方展开，再将 $\alpha_i^{(n)}$ 的表达式代入展开后的 E 得到：

$$E = \frac{1}{N}(\sum_{i=1}^N ||\mathbf{x}^{(n)} - \overline{\mathbf{x}}||^2 - 2\sum_{n=1}^N\sum_{i=1}^M \alpha_i^{(n)}(\mathbf{x}^{(n)} - \overline{\mathbf{x}})^T\mathbf{u}_i + \sum_{n=1}^N\sum_{i=1}^M(\alpha_i^{(n)})^2)$$
$$E = \frac{1}{N}\sum_{i=1}^N ||\mathbf{x}^{(n)} - \overline{\mathbf{x}}||^2 - \sum_{i=1}^M \mathbf{u}_i^T\frac{1}{N}\sum_{n=1}^N(\mathbf{x}^{(n)} - \overline{\mathbf{x}})(\mathbf{x}^{(n)} - \overline{\mathbf{x}})^T\mathbf{u}_i$$

     我们令 $s = \frac{1}{N}\sum_{n=1}^N(\mathbf{x}^{(n)} - \overline{\mathbf{x}})(\mathbf{x}^{(n)} - \overline{\mathbf{x}})^T$，$E$ 的前半部分是常数
  4. 将 E 写成矩阵形式：

$$E = ||\mathbf{X} - \overline{\mathbf{X}}||_F^2 - \sum_{i=1}^M \mathbf{u}_i^T\mathbf{S}\mathbf{u}_i$$

     其中：$\mathbf{X} = [\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \ldots, \mathbf{x}^{(N)}]$，$||\cdot||_F$ 是 Frobenius范数，定义为矩阵各项元素的绝对值平方的总和开根。
  5. 问题转换：
     - 在 $M = 1$ 的情况下，使用拉格朗日方法，可以看出 u_1 应该是 S 最大特征值对应的特征向量

$$\max_{u_1} u_1^T S u_1$$
$$\text{s.t. } u_1^T u_1 = 1$$
$$\text{Lagrange Method: } L = u_1^T S u_1 - \lambda(u_1^T u_1 - 1)$$
$$\text{derivative of u1} = 0\text{: } Su_1 = \lambda u_1$$

     - 在 $M = k$ 的情况下（推导同样使用拉格朗日方法，注意利用正交的条件），u_i **就是 S 最大的 k 个特征值对应的特征向量**

- **从最大化方差的角度理解 PCA**：最大化方差等于尽可能保留原始数据的信息
  - 推导过程：
    - 以 $M=1$ 为例：
      - **对于第一个方向 u_1，我们希望投影到 u_1 方 向上的数据方差，即 $\mathbf{u}_1^T\mathbf{x}^{(n)}$，是最大的。**
      - 方差的表达式：

$$
\begin{aligned}
var &= \frac{1}{N}\sum_{n=1}^{N}(\mathbf{u}_1^T(\mathbf{x}^{(n)}-\overline{\mathbf{x}}))^2 \\
&= \mathbf{u}_1^T\frac{1}{N}\sum_{n=1}^{N}(\mathbf{x}^{(n)}-\overline{\mathbf{x}})(\mathbf{x}^{(n)}-\overline{\mathbf{x}})^T\mathbf{u}_1 \\
&= \mathbf{u}_1^T\mathbf{S}\mathbf{u}_1
\end{aligned}
$$

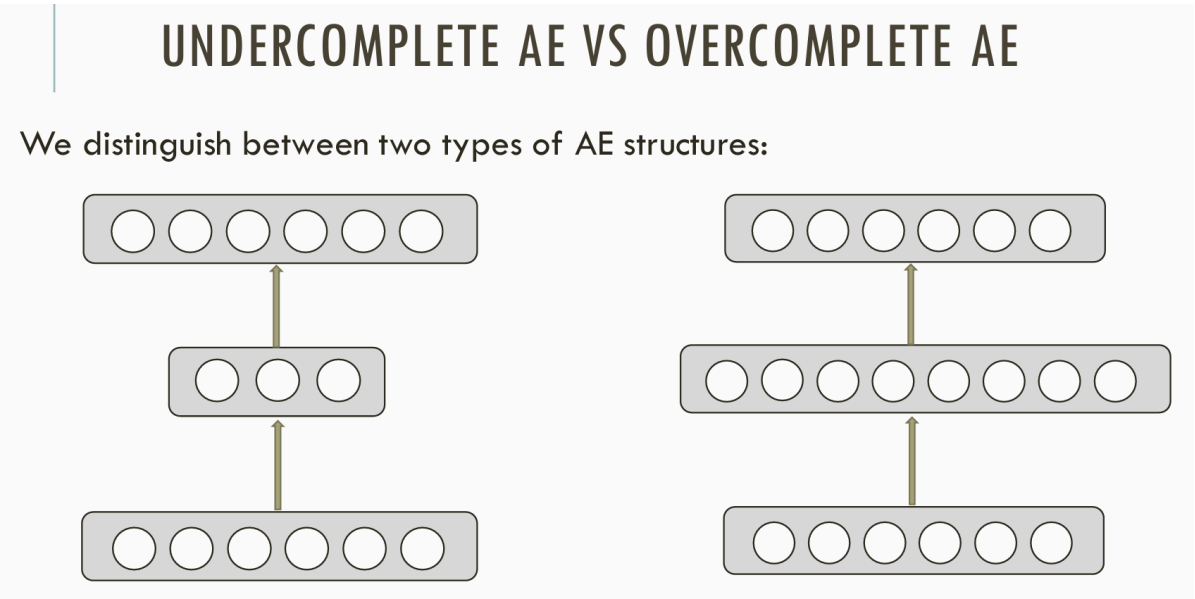
      - 同样可以推出 u_1 应该是 S 最大特征值对应的特征向量
    - 在 $M=k$ 的情况下（推导同样使用拉格朗日方法，注意利用正交的条件），u_i **就是 S 最大的 k 个特征值对应的特征向量**

4. 自动编码器 AE

是一种无监督机器学习算法。

基本上分为编码和解码过程：

- 编码：将输入通过一个编码器（通常是一个神经网络）来转换为低维的隐层表示。编码后的结果可以理解成输入数据的关键信息或者是一个隐含信息（隐含层）。
- 解码：将隐层表示通过解码器进行重构，希望能恢复出原数据。目标是让解码器输出尽可能接近输入数据。



目标函数：

自动编码器的训练目标是最小化输入数据和重建数据之间的误差。常见的损失函数有：

- **均方误差（MSE, Mean Squared Error）**：对于连续值数据，最常用的损失函数是均方误差：

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2$$

  其中，x_i 是输入数据，$\hat{x}_i$ 是重建数据。

- **交叉熵（Cross-Entropy）**：对于离散数据（如分类任务中的标签），可以使用交叉熵作为损失函数。

PCA与自动编码器的关系：

- 都是无监督的、降维、提取数据特征。
- 并且可以从降维后的数据尽量恢复出原始数据。
- **PCA只能处理线性数据，AE可以处理非线性数据。**

自动编码器的变种算法：

- **去噪自动编码器（Denoising Autoencoder, DAE）**： 输入数据中加入噪声，训练网络恢复原始的无噪声数据。这种方法可以帮助网络学习到更加鲁棒的特征。
- **变分自动编码器（Variational Autoencoder, VAE）**： 变分自动编码器是一种生成模型，它通过对潜在变量的概率分布进行建模来生成新的数据样本。

应用：

- 去噪
- 生成图片（VAE）

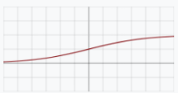
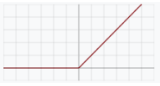
5. BP神经网络

主要作用是通过计算误差并将其传播回网络，从而更新神经网络的权重，使得网络的输出更接近目标值。

激活函数：

激活函数的作用是使得神经网络可以处理非线性的数据（想想看感知机处理异或）

- sigmoid好处是跟概率一样都是0到1，导数很好求，但是可能会梯度消失。
- Relu好处是不会梯度消失，并且计算更快，但是可能导致部分神经元（死神经元）一直不参与运算。

Logistic (a.k.a. Sigmoid or Soft step)		$f(x) = \sigma(x) = \frac{1}{1+e^{-x}}$ [1]	$f'(x) = f(x)(1-f(x))$
Rectified linear unit (ReLU)[15]		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$

[BP（反向传播算法）公式推导及例题解析 - 知乎]

前向和反向传播公式推导一下。

基于动量的梯度下降法：

很像下降之后还有一个惯性，利用了更多的历史信息（对比经典梯度下降）。

1. **计算梯度的动量：**

$$v(t) = \beta v(t-1) + (1-\beta)\nabla E(\mathbf{w})$$

其中：

- $v(t-1)$ 是上一次的动量，
- β 是动量的超参数，通常取值接近于1（如0.9、0.99），它控制历史梯度在更新中的权重。

2. **更新参数：**

$$\mathbf{w}(t+1) = \mathbf{w}(t) - \eta v(t)$$

其中，$\mathbf{w}(t)$ 是当前的权重，η 是学习率，$v(t)$ 是包含了动量的梯度。

应用：

- 图像分类
- 语音识别

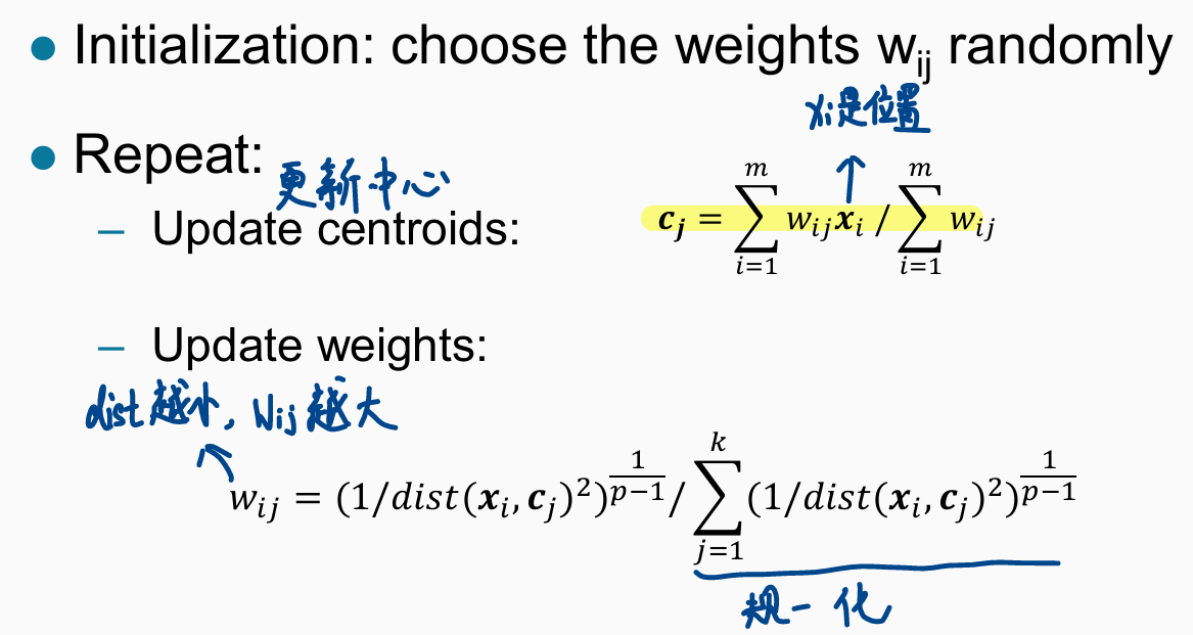
6. 聚类

分为软聚类和硬聚类。

软聚类表示数据点可以属于多个类，利用概率来表示。

硬聚类表示数据点只能属于一个类。

模糊C-means 是一个**软聚类**算法。



- Wij 表示数据点Xi属于聚类Cj的概率。
- 先固定Wij，来更新聚类中心点Cj。
- 再固定Cj，来更新权重Wij。

高斯混合聚类

一种基于概率的算法，是软聚类。

GMM假设数据是由若干个高斯分布混合而成，每个高斯分布代表一个簇。

Probabilistic Clustering: EM Algorithm

期望 max

参数(如正态均值,方差)

每个聚类是从一个分布表示，每个点是从一个分布采样

Initialize the parameters

Repeat

属于每个分布的概率 分布

① For each point, compute its probability under each distribution

② Using these probabilities, update the parameters of each distribution

Until there is no change

使用期望最大化（EM）算法进行优化：

- GMM通常使用期望最大化（EM）算法来估计模型参数。EM算法由两部分组成：
  - **E步（期望步）**：计算每个数据点属于每个高斯分布的后验概率（责任值）。

- E-step ("Expectation")
  - For each datum (example) x_i,
  - Compute "r_{ic}", the probability that it belongs to cluster c
    - Compute its probability under model c
    - Normalize to sum to one (over clusters c)

均值 方差

$$r_{ic} = \frac{\pi_c \mathcal{N}(x_i \; ; \; \mu_c, \Sigma_c)}{\sum_{c'} \pi_{c'} \mathcal{N}(x_i \; ; \; \mu_{c'}, \Sigma_{c'})}$$

样本i到聚类c的概率

  - If x_I is very likely under the cth Gaussian, it gets high weight
  - Denominator just makes r's sum to one

  - **M步（最大化步）**：基于责任值更新每个高斯分布的参数（均值、协方差和权重）。

- M-step ("Maximization")
  - For each cluster (Gaussian) x_c,
  - Update its parameters using the (weighted) data points

聚类c中样本总数

$$N_c = \sum_i r_{ic}$$ **Total responsibility allocated to cluster c**

$$\pi_c = \frac{N_c}{N}$$ **Fraction of total assigned to cluster c**

样本总数

$$\mu_c = \frac{1}{N_c} \sum_i r_{ic} x_i$$

更新均值

Weighted mean of assigned data

$$\Sigma_c = \frac{1}{N_c} \sum_i r_{ic} (x_i - \mu_c)^T (x_i - \mu_c)$$

更新方差

Weighted covariance of assigned data (use new weighted means here)

K Means聚类

> 目的，终止条件，一定最优吗，kmeans有哪几步，优缺点，乘积聚类。

无监督、硬聚类算法。

聚类的目的：提高类内的相似性，减少类间相似性。

Kmeans 流程如下：

1. 一开始随机初始化k个中心点。
2. 计算每个数据点到每个中心点的距离，选择最小的中心点的类作为数据点的类。
3. 计算类间位置均值，将该均值调整为该类新的中心点。
4. 重复上述步骤2 3，直到收敛。

Kmeans目标函数：

K-Means算法的核心目标是最小化每个簇内数据点与簇中心之间的距离。具体地，K-Means通过以下目标函数来度量聚类效果：

$$SSE = \sum_{i=1}^{K} \sum_{x_j \in C_i} ||x_j - \mu_i||^2$$

不一定最优：

- 因为Kmeans很吃初始点的位置，如果选择的不好对最后的分类结果影响很大。
- 并且如果数据点的分布很复杂，Kmeans效果也不太好，Kmeans假设簇是圆形的。
- K值的取值也是一个关键因素。

改进：选择中心位置的时候，先假设选择前k个中心点，剩下的中心点距离前k个中心点越远越好。

算法收敛的证明：

因为数据点到最优中心点的距离总和有下界，并且每次随着算法运行距离总和都会下降，最终收敛。

优缺点：

- 简单易懂
- 适合用于数据分布为球状的时候
- 对初始中心位置敏感
- 不能处理复杂形状的数据
- K值的取值也是一个关键因素。（选择肘部点）

单链接层次聚类

主要步骤：

1. **初始化**：每个数据点都看作一个单独的簇。
2. **计算距离**：计算所有簇之间的距离，初始时簇间的距离是簇内点之间的距离。对于每一对簇，计算簇间的距离。
3. **合并簇**：选择距离最小的两个簇进行合并，形成一个新的簇。
4. **更新距离**：更新新合并簇与其他簇之间的距离。使用**单链接**方法，即新的簇与其他簇之间的距离是新簇与其他簇之间点对距离的**最小值**。

5. **重复**：重复步骤3和步骤4，直到所有数据点都被合并成一个簇为止，或者满足某个停止条件（例如，设定的簇数）。

适用于对簇的形状没有严格要求的情况，尤其是当簇的形状不规则

7. EM算法（高斯混合聚类）

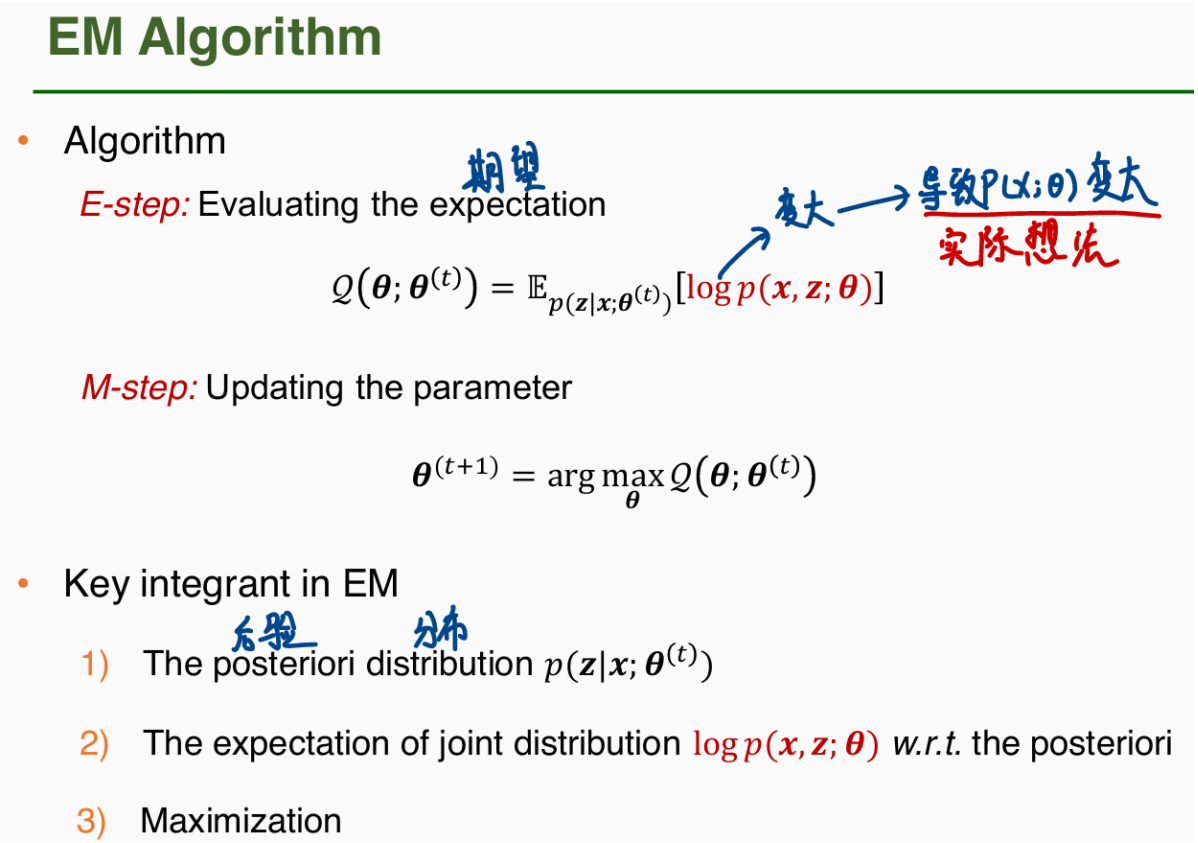
最大似然函数是根据X反推参数zeta，在高斯混合聚类中，参数就是聚类结果。

适合有观察变量和隐变量使用的算法。

为什么不是全局最优：

- EM算法的每一步（E步和M步）都是为了**局部改进**对数似然值，缺乏探索整个参数空间的能力。换句话说，它是一个贪心的算法，每次优化只考虑当前步骤的改进，而不是全局视角。
- 并且对初始参数敏感，如果设置的不好效果差很多。
- 所以容易陷入局部最优解。
- 试多几次。

介绍：



- **无监督分类问题，有点像 K-Means**
- **问题的一般形式**：给定一个联合分布 $p(x,z;\theta)$（在概率模型的参数 θ 下，x,z 的联合概率分布），x 是观测变量，z 是隐变量（Latent variable），目标是最大化似然分布：$\theta = \arg\max_\theta \log p(x;\theta)$，满足 $p(x;\theta) = \sum_z p(x,z;\theta)$。即我们需要根据联合概率分布函数 $p(x,z;\theta)$，优化边缘概率分布函数 $p(x;\theta)$

- **EM 算法（期望最大化算法）算法步骤**

  ## 总结E步和M步

  1. **E步：估计隐变量**
     - 计算隐变量 Z 的后验分布 $P(Z|X;\theta^{(t)})$，在当前模型参数下“猜测”隐变量的值。
     - 构造 $Q(\theta|\theta^{(t)})$ 函数，用于最大化对数似然函数的下界。
  2. **M步：优化模型参数**
     - 在 $Q(\theta|\theta^{(t)})$ 的基础上，优化模型参数 θ，使 $Q(\theta|\theta^{(t)})$ 最大化。
     - 更新模型参数为新的 $\theta^{(t+1)}$。

  - **E 步**：评估期望（下标所示分布下，括号内的随机变量的期望）
    - 假定参数已知，计算此时隐变量的后验概率（**求出每一个样本属于类别的期望值**）

$$Q(\theta;\theta^{(t)}) = \mathbb{E}_{p(z|x;\theta^{(t)})}[\log p(x,z;\theta)]$$

  - **M 步**：更新参数
    - 带入隐变量的后验概率，最大化样本分布的对数似然函数，求解相应的参数（**通过当前数据求出最可能的分布参数**）

$$\theta^{(t+1)} = \arg\max_{\theta} Q(\theta;\theta^{(t)})$$

  - 参数说明：
    - $p(z|x;\theta^{(t)})$：隐变量的后验分布（条件概率）
    - Q：联合分布的对数 $\log p(x,z;\theta)$ 关于后验概率分布 $p(z|x;\theta^{(t)})$ 的期望

    > 条件概率：$P(B|A) = \frac{P(AB)}{P(A)}$；
    > 全概率公式：$P(A) = \sum_{i=1}^{n} P(B_i)P(A|B_i)$

- EM 的证明：证明算法可以保证每一步的 likelihood 增加

$$\begin{aligned}\log p(x;\theta) &= \sum_z q(z)\log\frac{p(x,z;\theta)}{p(z|x;\theta)} \\ &= \sum_z q(z)\log\frac{p(x,z;\theta)}{q(z)} + \sum_z q(z)\log\frac{q(z)}{p(z|x;\theta)} \\ &= L(q,\theta) + KL(q||p(z|x;\theta)), \forall\theta, q(z)\end{aligned}$$

  - 其中：$KL(q||p) = \int q(z)\log\frac{q(z)}{p(z)}dz$ 是 KL 散度，用于衡量两个分布的距离

$$L(p(z|x;\theta^{(t)}),\theta^{(t)}) = \sum_z p(z|x;\theta^{(t)})\log\frac{p(x,z;\theta^{(t)})}{p(x|z;\theta^{(t)})}$$

- **训练高斯混合模型的例子**
  - 混合高斯分布：

$$p(x) = \sum_{k=1}^{K}\pi_k\mathcal{N}(x;\mu_k,\Sigma_k)$$

    - 可以将其表示为联合分布的边缘分布（？）

$$p(x,z)=p(x|z)p(z)=\prod_{k=1}^{K}[\pi_k\mathcal{N}(x;\mu_k,\Sigma_k)]^{z_k}$$

其中，$\mathbf{z}=[z_1,z_2,\ldots,z_k]$ 服从分类分布，参数为 π

> 分类分布：$P(X=x_k\theta_1,\theta_2,\ldots\theta_K)=\prod_{k=1}^{K}\theta_k^{x_k}$，其中 $\sum_{k=1}^{K}\theta_k=1,x_k\in 0,1,k\in 1,2,\ldots,K$

- **E 步**：
  - 后验分布：其中 1_k 指第 k 个元素为 1 的独热向量

$$p(z=1_k|x;\theta)=\frac{p(x,z=1_k;\theta)}{\sum_{i=1}^{K}p(x,z=1_i;\theta)}$$

  - 对数联合分布：就直接在混合高斯分布上加 $\log$ 就行了

$$\log p(x,z;\theta)=\sum_{k=1}^{K}z_k\cdot[\log\mathcal{N}(x|\mu_k,\Sigma_k)+\log\pi_k]$$

  - 计算期望：

$$\begin{aligned}&\mathbb{E}_{p(z|x;\theta^{(t)})}[\log p(x,z;\theta)]\\=&\sum_{k=1}^{K}\mathbb{E}_{p(z|x;\theta^{(t)})}[z_k][\log\mathcal{N}(x|\mu_k,\Sigma_k)+\log\pi_k]\end{aligned}$$

    其中，$E_{p(z|x;\theta^{(t)})}[z_k]$ 用后验分布的式子代替（将混合高斯分布代入 p，参数加上标）

$$E_{p(z|x;\theta^{(t)})}[z_k]=\frac{\mathcal{N}(x|\mu_k^{(t)},\Sigma_k^{(t)})\pi_k}{\sum_{i=1}^{K}\mathcal{N}(x|\mu_i^{(t)},\Sigma_i^{(t)})\pi_i}=\gamma_k^{(t)}$$

    最后代入到 $Q(\theta;\theta^{(t)})$：

$$Q(\theta;\theta^{(t)})=\sum_{k=1}^{K}\gamma_k^{(t)}[\log\mathcal{N}(x|\mu_k,\Sigma_k)+\log\pi_k]$$

  - 将 $\mathcal{N}(x|\mu_k,\Sigma_k)=\frac{1}{(2\pi)^{D/2}|\sigma|^{1/2}}\exp\{-\frac{1}{2}(x-\mu_k)^T\Sigma_k^{-1}(x-\mu_k)\}$ 代入 $Q(\theta;\theta^{(t)})$：

$$Q(\theta;\theta^{(t)})=\sum_{k=1}^{K}\gamma_k^{(t)}[-\frac{1}{2}(x-\mu_k)^T\Sigma_k^{-1}(x-\mu_k)-\frac{1}{2}|\Sigma_k|+\log\pi_k]+C$$

    C 是常量（$\frac{1}{(2\pi)^{D/2}}$ 部分）
  - 最后，考虑所有样本：

$$Q(\theta;\theta^{(t)})=\frac{1}{N}\sum_{n=1}^{N}\sum_{k=1}^{K}\gamma_k^{(t)}[-\frac{1}{2}(x^{(n)}-\mu_k)^T\Sigma_k^{-1}(x^{(n)}-\mu_k)-\frac{1}{2}|\Sigma_k|+\log\pi_k]+C$$

- **M 步**：通过对 μ_k,Σ_k,π_k 求导并将它们设为零，得到最佳 θ：

$$\mu_k^{(t+1)} = \frac{1}{N_k}\sum_{n=1}^{N}\gamma_{nk}x_n$$

$$\Sigma_k^{(t+1)} = \frac{1}{N_k}\sum_{n=1}^{N}\gamma_{nk}(x_n-\mu_k^{(t+1)})(x_n-\mu_k^{(t+1)})^T$$

$$\pi_k^{(t+1)} = \frac{N_k}{N}$$

其中：$N_k = \sum_{n=1}^{N}\gamma_{nk}$，分配给第 k 类的样本数

- **训练高斯混合模型的EM算法总结：**
  - 给定当前估算值：$\{\mu_k, \Sigma_k, \pi_k\}_{k=1}^K$，更新 γ_{nk}：

$$\gamma_{nk} \leftarrow \frac{\mathcal{N}(x|\mu_k,\Sigma_k)\pi_k}{\sum_{i=1}^{K}\mathcal{N}(x|\mu_i,\Sigma_i)\pi_i}$$

  - 跟据 γ_{nk}，更新 μ_k, Σ_k, π_k

$$N_k \leftarrow \sum_{n=1}^{N}\gamma_{nk}$$

$$\mu_k \leftarrow \frac{1}{N_k}\sum_{n=1}^{N}\gamma_{nk}x_n$$

$$\Sigma_k \leftarrow \frac{1}{N_k}\sum_{n=1}^{N}\gamma_{nk}(x_n-\mu_k)(x_n-\mu_k)^T$$

$$\pi_k \leftarrow \frac{N_k}{N}$$

8. 集成模型

动机：希望可以利用多个弱分类器来组合出一个强分类器，这些弱分类器可能在数据集的某些地方表现很好，如果将在不同部分上表现良好的弱分类器结合在一起，就可以得到一个强分类器。

- 例如有些分类器比较会识别猫，有些比较会识别狗。

结合的方法：

- 使用投票机制（无权重），即利用多数赞成来决定分类器的结果。**bagging**
- 使用加权平均投票，即表现好的分类器获得更大的权重。**boosting**

目的：

- 提高模型准确率
- 增加鲁棒性
- 减少过拟合和欠拟合
- 增加模型的多样性

Bagging

基本流程如下：

1. 采用bootstrap方法对数据集采样（有放回采样）K个数据集。
2. 根据这K个数据集训练出K个决策树。
3. 最终使用**大多数投票机制**将这K个决策树合并为一个决策树（强分类器）。

Boosting

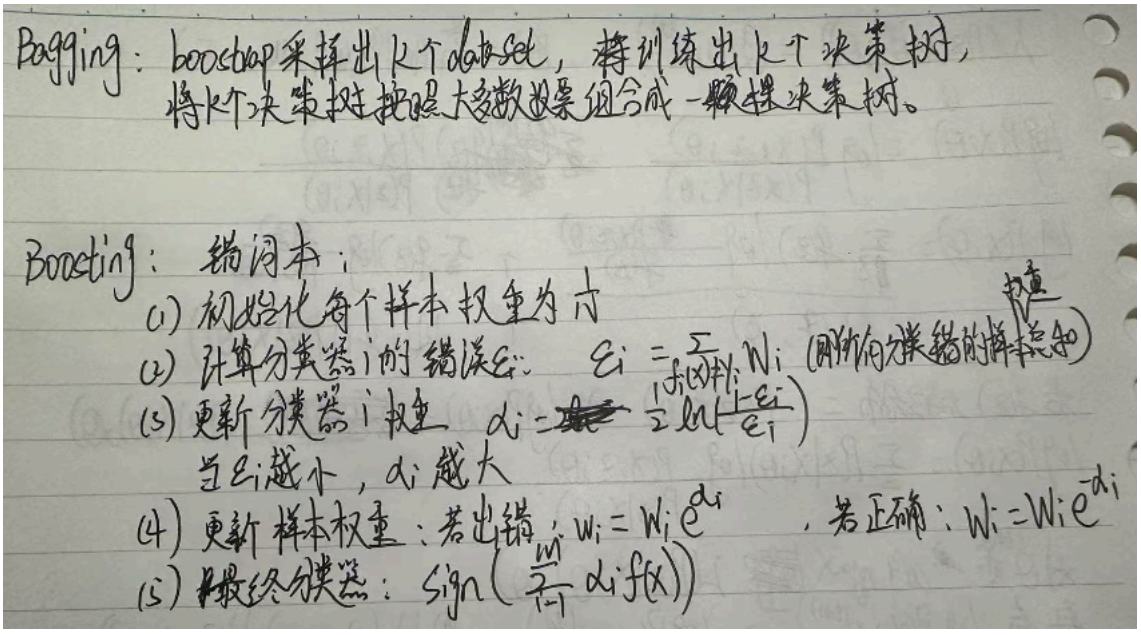
思想：如果样本A分错了，则下次采样到样本A的几率上升，很像错题本一样。

- **弱分类器创建：**重复执行以下步骤：识别被错误分类的例子，重新训练分类器，对错误分类的例子给予更多的权重 。
- **弱分类器合并：**将各分类器的预测结果进行**加权平均。**

Adaboost算法流程如下：

分类器准确率越高，权重越高。

- 迭代：计算第一个分类器 上的错误率，并为此分类器赋予权值 。最开始的时候，每一个样本的权重都是一样的，而现在需要将被错误分类的样本的权重以的倍率增强，被正确分类的样本的权重以的倍率减弱，不断迭代计算。
- 结合：每个弱分类器的输出乘以权重后加和做为最终输出。



4.3 bagging VS boosting：

■样本选择

- Bagging：训练集是在原始集中有放回选取的，从原始集中选出的各轮训练集之间是独立的
- Boosting：每一轮的训练集不变，只是训练集中每个样例在分类器中的权重发生变化。而权值是根据上一轮的分类结果进行调整

■样例权重

- Bagging： 使用均匀取样，每个样例的权重相等
- Boosting：根据错误率不断调整样例的权值，错误率越大则权重越大

■预测函数

- Bagging： 所有预测函数的权重相等
- Boosting：每个弱分类器都有相应的权重，对于分类误差小的分类器会有更大的权重

■并行计算

- Bagging： 各个预测函数可以并行生成
- Boosting：各个预测函数只能顺序生成，因为后一个模型参数需要前一轮模型的结果

无监督集成学习的挑战

例如考虑聚类，可能不同弱分类器分的聚类结果有不同的意思。

解决方法：

- 硬对应：标签重命名和投票。
- 软对应：软对应的思想是通过优化多个聚类结果之间的对应关系，找到一个共识聚类，使得每个数据点的归属在所有聚类中尽量一致。

  成员矩阵M的元素表示数据点与类别的归属关系。

  每个 Sj 矩阵是一个 n×n 的矩阵，记录了数据点之间的对应关系。

Minimize disagreement $\min \sum_{i=1}^{k} \| M - M_j S_j \|^2$

– $M_i S_i = M$

	C_1	C_2	C_3
v_1	1	3	2
v_2	1	3	2
v_3	2	1	2
v_4	2	1	3
v_5	3	2	1
v_6	3	2	1

本质上是一样的

$$\underbrace{\begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix}}_{M_2} \times \underbrace{\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}}_{S_2} = \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}}_{M}$$

9. Knn

这是个分类算法。

- 已经有一批有标签的数据了。
- 然后再输入一批没有标签的数据。
- 选取没有标签的数据的前k个最近邻居，统计它们的标签，出现最多的标签成为这个数据的标签。

计算距离公式

利用欧拉距离。

Euclidean distance (欧式距离)

$$d(p,q) = \sqrt{\sum_i (p_i - q_i)^2}$$

分类结果受什么影响

- **受到K值大小影响，如果k太小，容易受到噪声影响；如果k太大，则性能会下降，因为可能包括其他类别的邻居了。**
- 受原本有标签的样本数量影响，如果样本数量太小了，效果也不好。
- 需要对数据点进行归一化，因为需要计算距离，需要每个维度能表示的范围一致，避免被某个维度支配了。

如何确定k值

- k折交叉验证：

  将数据集划分为k等分，一份为测试集，其他为训练集，计算k次测试集上的正确率的平均值。比较knn不同的参数的k折交叉验证的结果，选取效果最好的。

优缺点

- 是懒惰算法，无需训练模型，可以适合动态数据。
- 原理简单，易实现。

- 对多分类问题很适合。
- 计算代价大，需要计算距离。
- k值选取需要谨慎。

10. 推荐系统

目的：目的是为了向用户推荐相关项，项可以是未看过的电影，网站或者是未买过的物品。推荐系统为客户实现一种个性化的推荐。

给定用户集合和项的集合，推出用户对项打分的集合。推荐系统可以总结为以下模型：

$$\text{Utility Function: } u : X \times S \to R$$

其中，X 是用户的集合，S 是项的集合，R 是用户对项评分的集合，并且是关于项的有序集。

- 推荐系统问题关键问题：如何为矩阵收集已知的评级，如何从已知的评级中推断未知的评级，如何评估推断的好坏。
- 冷启动问题：项没有被评分，用户的评分记录为空。

推荐系统有有三种实现方式：协同（需要打分，冷启动问题，相似用户或者相似商品）、基于内容特征（基于内容的特征，无需依赖其他用户，避免冷启动，但是比较吃用户的历史资料）、基于知识（无冷启动问题，难获得知识）。

三种实现的优缺点：

- 协同：需要进行打分，需要一个打分矩阵，容易有冷启动问题（对于基于用户的协同滤波算法，需要积累足够多的用户，并且用户有一定评分时才能找到一个用户的相似用户），主要思想是基于用户和商品的相似性来进行推荐，即假设用户A和用户B相似度很高，则推荐系统会认为它们喜欢的商品很接近，容易发现用户潜在的兴趣。不需要提取商品的特征，只依赖于用户的打分。有首评者问题，如果一个商品被打分的次数太少，那么也不会进行推荐；会有流行性传播的问题，一直推荐最近流行的商品，可能有用户不喜欢。
- 基于内容特征：注重于商品的特征，不需要依靠其他用户（无用户冷启动），可以实现比较精准的推荐，但是吃用户的历史资料并且难以获得商品的特征。如：一个人很喜欢看科幻电影，所以可以推荐科幻的书本给他。难以发现用户潜在的特征。
- 混合方法：可以利用加权平均组合协同和基于内容特征的推荐系统。例如将基于内容特征的特性加入到一个协同推荐系统中，当遇到用户冷启动（即用户刚刚打开网站），此时可以利用基于内容特征的推荐方法（利用用户的个人信息）为它推荐商品。

  例子：电影推荐系统，一个新用户协同推荐系统无法为他推荐电影，但是可以基于内容特征推荐系统根据这个新用户的年龄和性别对他进行推荐电影

> **物品-物品协同过滤方法和基于内容的推荐方法很相似**：这两者都通过评估物品之间的相似性来进行推荐。然而，为什么协同过滤方法没有物品特征选择的问题呢？
>
> 这个问题的意思是，物品-物品协同过滤和基于内容的方法在推荐逻辑上看起来有相似之处：它们都评估物品之间的相似性，然后推荐相似的物品。但与基于内容的方法需要选择合适的物品特征来进行比较不同，协同过滤方法则是基于用户的行为数据来计算物品之间的相似性，完全不需要考虑物品的具体特征。因此，协同过滤方法不受特征选择的困扰。

- **协同**
  - **基于用户的协同**

    思想：用户A和用户B如果对于过往的项的评分很接近，那么它们的喜好和兴趣是接近的。推荐系统为用户A提供一些与用户A相似度高的用户B的商品。

给定一个打分矩阵，计算用户a和用户b的相似度：

用户兴趣爱好相似度

- **A popular similarity measure in user-based CF: Pearson correlation**

$$sim(a,b) = \frac{\sum_{p\in P}(r_{a,p}-\bar{r}_a)(r_{b,p}-\bar{r}_b)}{\sqrt{\sum_{p\in P}(r_{a,p}-\bar{r}_a)^2}\sqrt{\sum_{p\in P}(r_{b,p}-\bar{r}_b)^2}}$$

度量 User a, User b 相似度

a, b : users

$r_{a,p}$: rating of user a for item p

P : set of items, rated both by a and b

Possible similarity values between -1 and 1; $\bar{r}_a, \bar{r}_b$ = user's average ratings

打分矩阵

	Item1	Item2	Item3	Item4	Item5
Alice	5	3	4	4	?
User1	3	1	2	3	3
User2	4	3	4	3	5
User3	3	3	1	5	4
User4	1	5	5	2	1

sim = 0,85
sim = 0,70
sim = -0,79

根据用户之间的相似性，可以预测用户对于这个项的打分是多少：

Making predictions

- A common prediction function:

$$pred(a,p) = \bar{r}_a + \frac{\sum_{b\in N} sim(a,b) * (r_{b,p} - \bar{r}_b)}{\sum_{b\in N} sim(a,b)}$$

预测 User a 在产品 P 上的打分

- Calculate, whether the neighbors' ratings for the unseen item ***i*** are higher or lower than their average
- Combine the rating differences – use the similarity as a weight
- Add/subtract the neighbors' bias from the active user's average and use this as a prediction

- **基于商品的协同**

思想：如果用户喜欢商品A同时也喜欢商品B，那么喜欢商品A的用户也很可能喜欢商品B。

计算商品之间的相似度：

$$sim(a,b) = \frac{\sum_{u\in U}(r_{u,a}-\overline{r_u})(r_{u,b}-\overline{r_u})}{\sqrt{\sum_{u\in U}(r_{u,a}-\overline{r_u})^2}\sqrt{\sum_{u\in U}(r_{u,b}-\overline{r_u})^2}}$$

对推荐系统的推荐进行评判的指标有：召回率，准确率，F1评判指标，均方根误差。

均方根误差（RMSE，Root Mean Squared Error）：计算用户评分与推荐系统预测评分之间的平方误差的平均值，并取平方根。公式如下：

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(r_{ui}-\hat{r}_{ui})^2}$$

- **记得看看基于内容的。**

11. 决策树

处理分类和回归问题，有监督学习算法。

基本思想：对属性进行逐层划分，每次划分将数据分割成不同的子集，以实现对目标的分类预测。

学习目的：为了实现一个泛化能力强的决策树。

决策树建立流程如下：

1. 根据算法选择 “最优属性” 。
2. 根据最优属性划分数据集。
3. 不断重复上述两个步骤，直到数据集内的数据都是同一类的。
4. 进行剪枝。

关键点在于**如何选择最优属性**进行划分。

- 希望能选择一个属性，划分完之后的数据集子集尽量都属于同一个类别。

有三种选择最优属性的判断方法：**信息熵**、**信息增益**、**基尼指数**。

ID3算法：使用信息增益选择测试属性。

$$H(x) = -\sum p_i log(p_i)$$

$$G(D,a) = H(D) - \sum_{v=1}^{V} \frac{D^v}{D} H(D^v)$$

信息增益倾向于取值较多的特征，因此我们对信息增益归一化（就是除以原本数据集D的熵），引入增益率：

每个西瓜有编号

若把“编号”也作为一个候选划分属性，则属性“编号”的信息增益为：

根结点的信息熵仍为： $Ent(D) = 0.998$

用“编号”将根结点划分后获得17个分支结点的信息熵均为：

所有分支熵都一样

$$Ent(D^1) = \boxed{?} = Ent(D^{17}) = -(\frac{1}{1}\log_2\frac{1}{1} + \frac{0}{1}\log_2\frac{0}{1}) = 0$$

则“编号”的信息增益为：

$$Gain(D,\text{编}\quad) = Ent(D) - \sum_{v=1}^{17}\frac{1}{17}Ent(D^v) = 0.998$$

远大于其他候选属性

信息增益准则对可取值数目较多的属性有所偏好

$$\text{Gain_ratio}(D,a) = \frac{\text{Gain}(D,a)}{\text{IV}(a)}$$

$$\text{IV}(a) = -\sum_{v=1}^{V} \frac{|D^v|}{|D|} \log_2 \frac{|D^v|}{|D|}$$

不过还存在的问题是，它更倾向于取值数目比较少的属性 。

启发式方法：一种启发式方法是结合二者，先从候选划分属性中找出信息增益高于平均水平的属性，再从中选取增益率最高的。

CART：使用gini指标进行属性选择

Gini值：反映了从D中随机抽取两个样本，其类别标记不一致的概率，因此Gini指标越小，代表这样分类越好。

$$\text{Gini}(D) = \sum_{k=1}^{|\mathcal{Y}|} \sum_{k' \neq k} p_k p_{k'} \quad = 1 - \sum_{k=1}^{|\mathcal{Y}|} p_k^2$$

$$\text{Gini_index}(D,a) = \sum_{v=1}^{V} \frac{|D^v|}{|D|} \text{Gini}(D^v)$$

决策树容易有**过拟合问题**：

1. **预剪枝：在模型构建过程中对模型进行剪枝**

   在决策树生成过程中，对每个结点在划分前进行验证集的估计，如果划分结点对泛化能力不带来提升，则停止划分并将当前结点记为叶结点。

   带来了欠拟合的风险。

2. **后剪枝：构建完之后再进行剪枝**

   后剪枝：先从训练集生成一棵完整的决策树，然后自底向上地对非叶结点进行考察，若将该结点对应的子树替换为叶结点能带来泛化性能提升则更换。

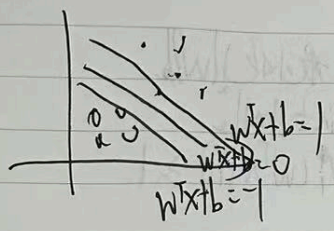
   泛化能力好，但是开销比预剪枝要大。

12. 支持向量机

目的：在空间中找出一个超平面，使得分类效果最好。找寻超平面的目标是最大化类内距离超平面最近的点（支持向量）的距离。

距离超平面最近的数据点成为支持向量，因为超平面是根据它们而选择的。

$w^Tx+b=0$ 的平面，使得支持向量对超平面距离大



$$m=\frac{|w^Tx+b|}{\sqrt{\|w\|_2^2}}=\frac{2}{\|w\|_2}$$

最大化 m 即最小化 $\|w\|$.

$$\boxed{\begin{array}{l}\text{目标函数 } \frac{1}{2}\|w\|^2 \\ s.t.\ y_i(w^Tx+b)\geq 1\end{array}}$$

根据等高线和向量均可知，若原函数为 $f(x,y)$，约束 $g(x,y)$

则当 $\nabla f(x,y)=\alpha\nabla g(x,y)$ 当二两个函数梯度平行时 $f(x,y)$ 取最小值，

即 $\nabla f(x,y)-\alpha\nabla g(x,y)=0$ 时，根据此特性构造 $L(w,b,\alpha)$ 函数：

$$L(w,b,\alpha)=\frac{1}{2}\|w\|^2-\sum_{i=1}^{m}\alpha_i\left[1-y_i(w^Tx+b)\right]$$

求关于 w 偏导：$\frac{\partial L}{\partial w}=\|w\|-\sum_{i=1}^{m}\alpha_i y_i x=0$，$w=\sum_{i=1}^{m}\alpha_i y_i x_i$

$\frac{\partial L}{\partial b}=\sum_{i=1}^{m}\alpha_i y_i=0$

将 $w=\sum_{i=1}^{m}\alpha_i y_i x_i$ 代入 $L(w,b,\alpha)$ 且约束 $\sum_{i=1}^{m}\alpha_i y_i=0$

此时该函数为 $\lambda(\alpha)=\ldots$

最大化该函数，求 α^*，代回 $w=\sum_{i=1}^{m}\alpha_i^* y_i$ 求出 w^*，根据 $y_i(w^{*T}x+b)=1$

可求出 b^*。

KTT条件：

① $\alpha_i\geq 0$

② $y_i(w^Tx+b)\geq 1$

③ $\alpha_i[y_i(w^Tx+b)-1]=0 \longrightarrow$ 若 $y_i(w^Tx+b)\neq 1$（非支持向量，$\alpha_i=0$）

若 $y_i(w^Tx+b)=1$（支持向量，$\alpha_i\neq 0$）

软间隔：如果数据集不是线性可分的，可以增加一部分松弛变量，使得模型允许一些分类错误。

- 其实我觉得可以把它想象成一个正则项。

软间隔分类的目标是最大化分类间隔，同时最小化分类错误。为了实现这一目标，我们需要引入松弛变量 ξ_i（即每个样本的松弛变量），这些松弛变量允许样本点位于间隔内或者被错误分类。

优化问题可以表示为：

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{N}\xi_i$$

其中：

- w 是超平面的法向量；
- b 是偏置；
- $\xi_i \geq 0$ 是第 i 个样本的松弛变量，表示该样本距离分类超平面的“错误程度”；
- C 是一个超参数，控制间隔的宽度和分类错误的权衡。

约束条件

对于每一个样本 i，其分类结果必须满足以下条件：

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \forall i$$

即：

- 如果样本点被正确分类，并且离超平面足够远，则 $\xi_i = 0$；
- 如果样本点被错误分类，或者距离超平面太近，则 $\xi_i > 0$。

核函数：因为支持向量机只能处理线性可分的数据，对于线性不可分的数据集可以使用核函数进行升维，升维度之后可能数据集就会可分了。

直接定义数据在高维空间的内积为核函数，不需要定义如何转换到高维空间。

记得高斯核。

$$\min_{\boldsymbol{\alpha}} \quad \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\alpha_i\alpha_j y_i y_j \boxed{\phi(\boldsymbol{x}_i)^\top \phi(\boldsymbol{x}_j)} - \sum_{i=1}^{m}\alpha_i$$

因此我们只需要定义核函数直接去表示这种内积

基本想法：不显式地设计核映射，而是设计核函数.

$$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \phi(\boldsymbol{x}_i)^\top \phi(\boldsymbol{x}_j)$$

名称	表达式	参数
线性核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \boldsymbol{x}_i^\top \boldsymbol{x}_j$	
多项式核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = (\boldsymbol{x}_i^\top \boldsymbol{x}_j)^d$	$d \geq 1$为多项式的次数
高斯核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \exp\left(-\frac{\lVert \boldsymbol{x}_i - \boldsymbol{x}_j \rVert^2}{2\delta^2}\right)$	$\delta > 0$为高斯核的带宽(width)
拉普拉斯核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \exp\left(-\frac{\lVert \boldsymbol{x}_i - \boldsymbol{x}_j \rVert}{\delta}\right)$	$\delta > 0$
Sigmoid核	$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \tanh(\beta \boldsymbol{x}_i^\top \boldsymbol{x}_j + \theta)$	tanh为双曲正切函数, $\beta > 0$, $\theta < 0$

优缺点:

- 能够处理高维分类
- 能处理非线性可分数据（经过核函数）
- 只依赖支持向量（一小部分数据集）
- 鲁棒性好

- 计算复杂度高
- 无法处理多分类问题